



CS6308 – JAVA PROGRAMMING

MINI PROJECT DOCUMENTATION

TOPIC: MINI CRYPTOGRAPHIC SIMULATOR

TEAM MEMBERS:

JANANIE . P (2021103017)

KRITHIVARSHA . I (2021103539)

MONICA . T (2021103543)

INDEX

CONTENTS	PAGE NO
ABSTRACT	3
PROBLEM DEFINITION	4
SOFTWARE DESCRIPTION	5
PROGRAM CODE	13
OUTPUT	25

ABSTRACT:

Mini Cryptographic simulator is a Java-based cryptographic application designed to provide a user-friendly interface for a variety of encryption, decryption, and hashing tasks. The program uses a JavaFX graphical user interface (GUI) to allow users to easily encrypt or decrypt text and files, as well as calculate hash values for both text and files to verify data integrity. Mini Cryptographic simulator supports a range of encryption algorithms, including AES128, AES256, DES and 3DES, along with several hashing algorithms such as MD2, MD5, SHA1, SHA224, SHA256, SHA384, and SHA512. With additional features like random key generation, drag-and-drop support, and a dark theme option, Mini Cryptographic simulator aims to offer a comprehensive cryptographic solution with a simple, user-friendly design.

PROBLEM DEFINITION:

In the age of digital information, the need for secure communication and data protection is more important than ever. Organizations and individuals alike require reliable encryption methods to ensure that sensitive data remains secure, whether it is stored locally, transmitted over a network, or shared with others. Additionally, verifying the integrity of files and data is critical to detect tampering or corruption. Traditional cryptographic tools can be complex and may require advanced technical knowledge, making them inaccessible to non-experts.

Mini Crypter addresses these issues by providing a straightforward and intuitive tool for encryption, decryption, and hashing. With its JavaFX-based GUI, users can perform a wide range of cryptographic operations without needing extensive technical expertise. By supporting popular encryption algorithms like AES and DES, along with a variety of hashing methods, the program offers flexibility and adaptability for various use cases. The user-friendly features makes Mini Cryptographic simulator a comprehensive solution for users seeking accessible and effective cryptographic functionality.

SOFTWARE DESCRIPTION:

BUILT-IN PACKAGES:

- **java.io**

  Contains classes for input/output operations in Java.

- **javafx.scene.control**

  Contains user interface components for JavaFX applications, referred to as "controls" and include elements like buttons, labels, text fields, combo boxes, menus, and alerts. The **Alert class**, in particular, is used to display pop-up messages, such as warnings, error messages, or information dialogs.

- **javafx.animation**

  Provides classes for creating animations in JavaFX. These animations can add visual effects to GUI components, enhancing the user experience. The **TranslateTransition class** allows you to animate the movement (translation) of a node along a specific path over a defined duration. This is useful for creating smooth transitions, sliding effects, or animated sequences in your JavaFX application.

- **javafx.scene**

  Contains the base classes for building scenes in a GUI application. A "scene" in JavaFX represents a collection of graphical elements displayed in a window. This package includes classes like Parent, which is a base class for all node containers, and Group, Node, Scene, which define various structures in a scene graph.

- **javafx.fxml**

  Provides support for the FXML markup language, which is used to define the structure and layout of JavaFX user interfaces. The **FXMLLoader class** is used to load FXML files and create JavaFX objects based on the defined layout and properties.

- **java.net**

  Contains classes for network-related operations, such as working with URLs and network connections. The **URL class** represents a Uniform Resource Locator, giving access to resources over the internet or from local files. It is used in JavaFX for tasks like loading FXML files from a specified location or fetching resources from a network source.

CUSTOM PACKAGES:

PACKAGE 1: Cipher

Contains all encryption algorithms used for text encryption/decryption. Each class in this package represents a specific encryption algorithm or a helper class for encryption-related tasks.

CLASSES:

Aes128:

Implements the AES (Advanced Encryption Standard) algorithm with a 128-bit key. Provides methods for encrypting and decrypting text and files using a specified key. Useful for secure, high-speed encryption.

Aes256:

Similar to AES128Cipher but uses a 256-bit key for stronger encryption. It is suitable for applications where a higher level of security is required.

Des:

Implements the DES (Data Encryption Standard) algorithm. Offers basic encryption and decryption methods. Although DES is considered less secure compared to AES, it is still used in some legacy systems.

Des3:

Implements the Triple DES (3DES) algorithm, which applies DES encryption three times in succession for enhanced security. Provides encryption and decryption methods for text and files.

CurrentCipher:

This class allows the application to switch between various cryptographic algorithms for text encryption, decryption, key management, and other related operations.

PACKAGE 2: GUI

SUBPACKAGE 1: Controllers

Comprises classes that handle the control logic for various GUI components. These controllers manage user interactions, update the view based on changes in the application state, and communicate with the appropriate business logic to execute application functions.

CLASSES:

AboutPageController:

This controller manages the "About" page of the application, which typically contains information about the software, such as its purpose, version, and developers. It may handle events like opening the page and interacting with any interactive elements within it.

FileHashPageController:

This controller handles the "File Hash" functionality, allowing users to calculate hash values for files. It likely manages file selection, initiates hash calculation, and displays the resulting hash value.

HashSumCheckerPageController:

This controller is responsible for the "Hash Sum Checker" functionality, where users can verify file integrity by comparing hash sums. It may manage user interactions to select files, calculate their hash values, and compare them with expected hash sums.

HomePageController:

This controller manages the "Home" or main page of the application. It likely serves as the central point of interaction, directing users to other sections and coordinating overall application navigation.

MenuBarController:

This controller manages the application's menu bar, handling user interactions like selecting menu items and triggering corresponding actions. It might include operations like opening different pages, accessing settings, or closing the application.

SettingsPageController:

This controller is responsible for the "Settings" page, where users can configure application preferences. It handles user interactions related to changing settings, saving preferences, and possibly restoring defaults.

ShowKeyController:

This controller manages the functionality to display encryption keys to the user. It may handle displaying the current key, copying it to the clipboard, or generating a new key.

SideMenuController:

This controller manages the side menu, which might provide navigation options to different sections of the application. It handles user interactions like selecting menu items to navigate to different pages.

TextEncryptionPageController:

This controller is responsible for the "Text Encryption" page, where users can encrypt and decrypt text. It manages user inputs, encryption/decryption operations, and the display of results.

TextHashPageController:

This controller handles the "Text Hash" functionality, allowing users to calculate hash values for text. It manages user inputs, initiates hash calculation, and displays the hash result.

UserKeyController:

This controller manages user-generated encryption keys. It handles creating, saving, and retrieving user-defined keys, and possibly allows users to set a custom key for encryption/decryption.

SUBPACKAGE 2: Main

Central organizational unit in a JavaFX-based application, housing essential components and utility classes that are core to the functioning of the graphical user interface (GUI).

CLASSES:

AlertDialog:

The AlertDialog class is responsible for displaying alert messages and notifications to users. This class typically encapsulates common dialog operations, such as showing error messages, warnings, information, or confirmations. It provides a consistent way to communicate with users about events, errors, or other situations that require user attention.

GUI:

The GUI class might represent the main entry point for the graphical user interface. It sets up the primary JavaFX stage and initializes key GUI components, such as the main window, menus, and other interface elements. This class might also manage the main application loop, controlling the overall flow and interaction with the GUI.

Pages:

The Pages class could act as a central point for managing different pages within the application. It might include methods to navigate between pages, load specific sections of the GUI, and control the visibility of various elements. This class is useful for maintaining a modular structure and allowing seamless navigation within the application.

ShowKey:

The ShowKey class likely represents the GUI component for displaying encryption keys. It might include methods to show, generate, or copy encryption keys, as well as user interaction handling related to these operations. This class could work closely with corresponding controller and CSS classes to ensure a smooth user experience.

UserKey:

The UserKey class could represent a section of the GUI dedicated to managing user-generated encryption keys. It might include methods for users to create, set, or retrieve custom keys for encryption and decryption. This class ensures that users have control over the key generation process and can maintain their own encryption keys.

PACKAGE 3: Hash

Hash-related operations, provide functionality to calculate hash values, verify hash sums, and perform other hashing-related tasks.

MD2:

MD2 (Message Digest Algorithm 2) It generates a 128-bit hash value, vulnerable to modern cryptographic attacks.

MD5:

MD5 (Message Digest Algorithm 5) also produces a 128-bit hash value widely used but not secure for cryptographic purposes.

SHA-1

SHA-1 (Secure Hash Algorithm 1) It generates a 160-bit hash value susceptible to collision attacks, being phased out for secure applications.

SHA-224

SHA-224 (Secure Hash Algorithm 224-bit) is a truncated version of SHA-256, producing a 224-bit hash value derived from SHA-256, more secure than SHA-1.

SHA-256

SHA-256 (Secure Hash Algorithm 256-bit) It produces a 256-bit hash value , widely used for secure applications, strong resistance to collision attacks.

SHA-384

SHA-384 (Secure Hash Algorithm 384-bit) is derived from SHA-512, truncated to produce a 384-bit hash value, stronger than SHA-256 but with reduced overhead compared to SHA-512.

SHA-512

SHA-512 (Secure Hash Algorithm 512-bit) produces a 512-bit hash value, provides high security, more computationally intensive than other SHA-2 variants.

CurrentHash

A central controller for hash-related operations in your application. It allows for flexibility in selecting and switching between different hash algorithms and provides methods for calculating hash values for text and files.

PROGRAM CODE:

GUI.java: This is the main executable class.

```
1  package GUI.Main;
2
3  import javafx.animation.TranslateTransition;
20
21 public class GUI extends Application {
22
23     private static BorderPane borderPane = new BorderPane();
24     private static Pane pane;
25     private static VBox menu;
26     private static TranslateTransition menuTranslation;
27
28     @Override
29     public void start(Stage primaryStage) {
30         try {
31             primaryStage.setTitle("MiniCrypter");
32             primaryStage.getIcons().add(new Image("/SJC1.png"));
33             initializePages();
34
35             Scene scene = new Scene(new Pane(), 590, 390);
36             Parent menuBar = FXMLLoader.load(getClass().getResource("/GUI/fxml/MenuBar.fxml"));
37
38             borderPane.setTop(menuBar);
39             pane = new Pane(Pages.HOME_PAGE.getParent());
40             pane.getStylesheets().add(getClass().getResource("/GUI/css/General.css").toExternalForm());
41             borderPane.setCenter(pane);
42
43             menu = FXMLLoader.load(getClass().getResource("/GUI/fxml/SideMenu.fxml"));
44
45             menu.setTranslateX(-200);
46             menuTranslation = new TranslateTransition(Duration.millis(250), menu);
47             menuTranslation.setFromX(-400);
48             menuTranslation.setToX(0);
49
50             menu.setOnMouseExited(evt -> {
51                 //bug here
52                 menuTranslation.setRate(-1);
53                 menuTranslation.play();
54             });
55
56             pane.getChildren().add(menu);
57             scene.setRoot(borderPane);
58             primaryStage.setResizable(false);
59             primaryStage.setScene(scene);
60             primaryStage.show();
61         } catch (IOException e) {
```

```
            e.printStackTrace();
        }
    }

    private void initializePages() {
        try {
            Pages.ABOUT_PAGE.setParent(FXMLLoader.load(getClass().getResource("/GUI/fxml/AboutPage.fxml")));
            Pages.FILE_ENCRYPTION_PAGE.setParent(FXMLLoader.load(getClass().getResource("/GUI/fxml/FileEncryptionPage.fxml")));
            Pages.FILE_HASH_PAGE.setParent(FXMLLoader.load(getClass().getResource("/GUI/fxml/FileHashPage.fxml")));
            Pages.HASH_SUM_CHECKER_PAGE.setParent(FXMLLoader.load(getClass().getResource("/GUI/fxml/HashSumCheckerPage.fxml")));
            Pages.HOME_PAGE.setParent(FXMLLoader.load(getClass().getResource("/GUI/fxml/HomePage.fxml")));
            Pages.SETTINGS_PAGE.setParent(FXMLLoader.load(getClass().getResource("/GUI/fxml/SettingsPage.fxml")));
            Pages.TEXT_ENCRYPTION_PAGE.setParent(FXMLLoader.load(getClass().getResource("/GUI/fxml/TextEncryptionPage.fxml")));
            Pages.TEXT_HASH_PAGE.setParent(FXMLLoader.load(getClass().getResource("/GUI/fxml/TextHashPage.fxml")));
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public static void updatePageContent(Node nextNode) {
        pane.getChildren().set(0, nextNode);
    }

    public static void hideMenu() {
        menuTranslation.setRate(-1);
        menuTranslation.play();
    }

    public static void menuButtonPressed() {
        menuTranslation.setRate(1);
        menuTranslation.play();
    }

    public static void main(String[] args) {
        launch(args);
    }

}
```

Fig 1: GUI.java

MenuBar.fxml: This is the fxml file through which all the other page are navigated.

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.Menu?>
<?import javafx.scene.control.MenuBar?>
<?import javafx.scene.control.MenuItem?>
<?import javafx.scene.control.RadioMenuItem?>

Bind to grammar/schema...
<MenuBar style="-fx-background-color: #ffffff;" xmlns="http://javafx.com/javafx" xmlns:fx="http://javafx.com/fxml" fx:controller=
    <Menu mnemonicParsing="false" text="File">
        <MenuItem mnemonicParsing="false" onAction="#homePageAction" text="Home" />

    </Menu>
    <Menu mnemonicParsing="false" text="Options">
        <MenuItem mnemonicParsing="false" onAction="#genKeyAction" text="Generate Key" />
        <MenuItem mnemonicParsing="false" onAction="#showKeyAction" text="Show key" />
        <MenuItem mnemonicParsing="false" onAction="#setKeyAction" text="Set Key" />
    </Menu>
    <Menu mnemonicParsing="false" text="Hash">
        <RadioMenuItem fx:id="MD2_radio" mnemonicParsing="false" onAction="#MD2_radioAction" text="MD2" />
        <RadioMenuItem fx:id="MD5_radio" mnemonicParsing="false" onAction="#MD5_radioAction" text="MD5" />
        <RadioMenuItem fx:id="SHA1_radio" mnemonicParsing="false" onAction="#SHA1_radioAction" text="SHA-1" />
        <RadioMenuItem fx:id="SHA224_radio" mnemonicParsing="false" onAction="#SHA224_radioAction" text="SHA-224" />
        <RadioMenuItem fx:id="SHA256_radio" mnemonicParsing="false" onAction="#SHA256_radioAction" text="SHA-256" />
        <RadioMenuItem fx:id="SHA384_radio" mnemonicParsing="false" onAction="#SHA384_radioAction" text="SHA-384" />
        <RadioMenuItem fx:id="SHA512_radio" mnemonicParsing="false" onAction="#SHA512_radioAction" text="SHA-512" />
    </Menu>
    <Menu mnemonicParsing="false" text="Ciphers">
      <RadioMenuItem fx:id="DES_radio" mnemonicParsing="false" onAction="#DES_radioAction" text="DES" />
      <RadioMenuItem fx:id="DES3_radio" mnemonicParsing="false" onAction="#DES3_radioAction" text="3DES" />
        <RadioMenuItem fx:id="AES128_radio" mnemonicParsing="false" onAction="#AES128_radioAction" text="AES-128" />
        <RadioMenuItem fx:id="AES256_radio" mnemonicParsing="false" onAction="#AES256_radioAction" text="AES-256" />


    </Menu>
    <Menu mnemonicParsing="false" text="Help">
        <MenuItem mnemonicParsing="false" onAction="#aboutAction" text="About" />
    </Menu>
</MenuBar>

```

Fig 2: MenuBar.fxml

TextEncryptionController.java:

```
3  import Cipher.CurrentCipher;
12
13 public class TextEncryptionPageController {
14
15
16
17     public ImageView menuImage;
18
19     @FXML
20     private TextArea inputText;
21
22     @FXML
23     private TextArea outputText;
24
25
26
27     public void encrypt() {
28         outputText.setText(CurrentCipher.encrypt(inputText.getText()));
29
30     }
31
32     public void decrypt() {
33         outputText.setText(CurrentCipher.decrypt(inputText.getText()));
34     }
35
36     public void initialize() {
37         inputText.setPromptText("Input text");
38         inputText.setFocusTraversable(false);
39         outputText.setPromptText("Output text");
40
41     }
42
43     public void menuButtonPressed() {
44         menuImage.setImage(new javafx.scene.image.Image("/menu2.png"));
45         GUI.menuButtonPressed();
46     }
47
48     public void menuButtonRelease() {
49         menuImage.setImage(new javafx.scene.image.Image("/menu1.png"));
50     }
51
52     public void mouseEntered() {
53         menuImage.setImage(new Image("/menu2.png"));
54     }
55
```

Fig 3: TextEncryptionController.java

FileHashPageController.java

```java
package GUI.Controllers;

import GUI.Main.GUI;

public class FileHashPageController {

    public ImageView menuImage;

    @FXML
    public Button browseInFile;

    @FXML
    public Text fileInPath;

    @FXML
    public Text dragNDropText;

    @FXML
    public TextArea hashSumText;

    private boolean fileInReady = false;
    private File in;

    public void browseInButtonAction() {
        FileChooser fileChooser = new FileChooser();
        fileChooser.setTitle("Select file");
        in = fileChooser.showOpenDialog(browseInFile.getScene().getWindow());
        //debug here
        if (in==null) {
            fileInReady = false;
            return;
        }
        hideDragDropText();
        fileInPath.setText(PathCutter.cutPath(in.getPath()));
        fileInReady = true;
    }

    public void handleDragOver(DragEvent event) {
        if (event.getDragboard().hasFiles()) {
            event.acceptTransferModes(TransferMode.ANY);
        }
    }

    public void handleDragDropped(DragEvent event) {
        List<File> files = event.getDragboard().getFiles();
        in = files.get(0);
        hideDragDropText();
        fileInPath.setText(PathCutter.cutPath(in.getPath()));
        fileInReady = true;
    }

    public void hashButtonAction() {
        if (fileInReady) {
            hashSumText.setText(CurrentHash.fileHashSum(in));
        }
    }

    private void hideDragDropText() {
        dragNDropText.setVisible(false);
    }

    private void enableDragDropText() {
        dragNDropText.setDisable(true);
    }

    public void menuButtonPressed() {
        menuImage.setImage(new javafx.scene.image.Image("/menu2.png"));
        GUI.menuButtonPressed();
    }

    public void menuButtonRelease() {
        menuImage.setImage(new javafx.scene.image.Image("/menu1.png"));
    }

    public void mouseEntered() {
        menuImage.setImage(new Image("/menu2.png"));
    }

}
```

Fig 4: FileHashPageController.java

HashSumCheckerPageController.java

```java
package GUI.Controllers;

import GUI.Main.GUI;

public class HashSumCheckerPageController {

    @FXML
    public Button browseInFile;

    @FXML
    public Text fileInPath;

    @FXML
    public Text dragNDropText;

    @FXML
    public TextArea hash2;

    private File in;
    private boolean fileInReady = false;
    public ImageView menuImage;
    public ImageView operationStatus;

    public void verifyChecksum() {
        if (fileInReady && hash2.getText()!=null) {
            if (hash2.getText().equals(CurrentHash.fileHashSum(in))) {
                operationStatus.setImage(new Image("/d30.png"));
            } else {
                operationStatus.setImage(new Image("/e30.png"));
            }
        }
    }

    public void browseInButtonAction() {
        FileChooser fileChooser = new FileChooser();
        fileChooser.setTitle("Select file");
        in = fileChooser.showOpenDialog(browseInFile.getScene().getWindow());
        //Debug here
        if (in==null) {
            fileInReady = false;
            return;
        }
        hideDragDropText();
        fileInPath.setText(PathCutter.cutPath(in.getPath()));
        fileInReady = true;
```

```
60      }
61
62      public void handleDragOver(DragEvent event) {
63          if (event.getDragboard().hasFiles()) {
64              event.acceptTransferModes(TransferMode.ANY);
65          }
66      }
67
68      public void handleDragDropped(DragEvent event) {
69          List<File> files = event.getDragboard().getFiles();
70          in = files.get(0);
71          hideDragDropText();
72          fileInPath.setText(PathCutter.cutPath(in.getPath()));
73          fileInReady = true;
74      }
75
76      private void hideDragDropText() {
77          dragNDropText.setVisible(false);
78      }
79
80      private void enableDragDropText() {
81          dragNDropText.setDisable(true);
82      }
83
84      public void menuButtonPressed() {
85          menuImage.setImage(new Image("/menu2.png"));
86          GUI.menuButtonPressed();
87      }
88
89      public void menuButtonRelease() {
90          menuImage.setImage(new Image("/menu1.png"));
91      }
92
93      public void mouseEntered() {
94          menuImage.setImage(new Image("/menu2.png"));
95      }
96
97  }
```

Fig 5: HashSumCheckerPageController.java

TextHashPageController.java

```java
package GUI.Controllers;

import GUI.Main.GUI;

public class TextHashPageController {

    public ImageView menuImage;

    @FXML
    private TextArea inputText;

    @FXML
    private TextArea outputText;

    public void initialize() {
        inputText.setPromptText("Input text");
        inputText.setFocusTraversable(false);
        outputText.setPromptText("Output text");
    }

    public void hashButtonAction() {
        outputText.setText(CurrentHash.hashText(inputText.getText()));
    }

    public void menuButtonPressed() {
        menuImage.setImage(new javafx.scene.image.Image("/menu2.png"));
        GUI.menuButtonPressed();
    }

    public void menuButtonRelease() {
        menuImage.setImage(new javafx.scene.image.Image("/menu1.png"));
    }

    public void mouseEntered() {
        menuImage.setImage(new Image("/menu2.png"));
    }

}
```

Fig 6:TextHashPageController.java

SideMenuController.java:

```java
package GUI.Controllers;

import GUI.Main.GUI;

public class SideMenuController {

    public void homeButtonAction() {
        GUI.updatePageContent(Pages.HOME_PAGE.getParent());
        GUI.hideMenu();
    }

    public void hashSumCheckerButtonAction() {
        GUI.updatePageContent(Pages.HASH_SUM_CHECKER_PAGE.getParent());
        GUI.hideMenu();
    }

    public void textEncryptionButtonAction() {
        GUI.updatePageContent(Pages.TEXT_ENCRYPTION_PAGE.getParent());
        GUI.hideMenu();
    }

    public void fileEncryptionButtonAction() {
        GUI.updatePageContent(Pages.FILE_ENCRYPTION_PAGE.getParent());
        GUI.hideMenu();
    }

    public void fileHashButtonAction() {
        GUI.updatePageContent(Pages.FILE_HASH_PAGE.getParent());
        GUI.hideMenu();
    }

    public void textHashButtonAction() {
        GUI.updatePageContent(Pages.TEXT_HASH_PAGE.getParent());
        GUI.hideMenu();
    }

    public void aboutButtonAction() {
        GUI.updatePageContent(Pages.ABOUT_PAGE.getParent());
        GUI.hideMenu();
    }

}
```

Fig 7: SideMenuController.java

ShowKeyController.java:

```java
package GUI.Controllers;

import Cipher.CurrentCipher;

public class ShowKeyController {

    @FXML
    private Text cipherName;

    @FXML
    private TextField keyField;

    public void initialize() {
        cipherName.setText(CurrentCipher.getCurrentCipherName()+" KEY");
        keyField.setText(CurrentCipher.getKey());
    }

}
```

Fig 8: ShowKeyController.java

UserKeyController.java:

```java
package GUI.Controllers;

import Cipher.CurrentCipher;

public class UserKeyController {

    @FXML
    private Text cipherName;

    @FXML
    private TextField userKeyField;

    public void initialize() {
        cipherName.setText("Enter "+CurrentCipher.getCurrentCipherName()+" key: ");
    }

    public void enterKey() {
        String userKey = userKeyField.getText();
        CurrentCipher.setKey(userKey);
        Stage stage = (Stage) userKeyField.getScene().getWindow();
        stage.close();
    }

}
```

Fig 9: UserKeyController.java

AES128.java

```
package Cipher;

import GUI.Main.AlertDialog;

class Aes128 {

    private SecretKey key;
    private Cipher cipher;
    private IvParameterSpec ivParameterSpec;

    Aes128() {
        try {
            cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
        } catch (NoSuchAlgorithmException | NoSuchPaddingException e) {
            e.printStackTrace();
        }
        generateIV();
        generateKey();
    }

    String encryptString(String text) {
        try {
            cipher.init(Cipher.ENCRYPT_MODE, key, ivParameterSpec);
            return Base64.getEncoder().encodeToString(cipher.doFinal(text.getBytes(StandardCharsets.UTF_8)));
        } catch (InvalidKeyException | IllegalBlockSizeException | BadPaddingException | InvalidAlgorithmParameterException e) {
            e.printStackTrace();
            AlertDialog.showError("AES-128 text encryption error", e.toString());
            return null;
        }
    }

    String decryptString(String encryptedText) {
        byte[] data;
        try {
            data = Base64.getDecoder().decode(encryptedText.getBytes(StandardCharsets.UTF_8));
        } catch (IllegalArgumentException e) {
            System.out.println("IllegalArgumentException caught");
            data = Base64.getDecoder().decode(RepairString.repairString(encryptedText).getBytes(StandardCharsets.UTF_8));
        }
        try {
            cipher.init(Cipher.DECRYPT_MODE, key, ivParameterSpec);
            return new String(cipher.doFinal(data), StandardCharsets.UTF_8);
        } catch (InvalidKeyException | IllegalBlockSizeException | BadPaddingException | InvalidAlgorithmParameterException e) {
            e.printStackTrace();
            AlertDialog.showError("Error occurred while decrypting text", e.toString());
            return null;
        }
    }

    void encryptFile(File in, File out) {
        try {
            cipher.init(Cipher.ENCRYPT_MODE, key, ivParameterSpec);
        } catch (InvalidKeyException | InvalidAlgorithmParameterException e) {
            e.printStackTrace();
            AlertDialog.showError("AES-128 init error!", e.toString());
            return;
        }
        byte[] iv = cipher.getIV();
        byte[] buffer = new byte[2048];
        try {
            FileOutputStream fileOut = new FileOutputStream(out);
            FileInputStream fileIn = new FileInputStream(in);
            CipherOutputStream cipherOut = new CipherOutputStream(fileOut, cipher);
            fileOut.write(iv);
            while (fileIn.read(buffer, 0, buffer.length)!=-1) {
                cipherOut.write(buffer, 0, buffer.length);
            }
            fileIn.close();
            fileOut.close();
            cipherOut.close();
            AlertDialog.showInfo("File was successfully encrypted!");
        } catch (IOException e) {
            e.printStackTrace();
            AlertDialog.showError("AES-128 file encryption error!", e.toString());
        }
    }

    void decryptFile(File in, File out) {
        IvParameterSpec parameterSpec;
        try {
            byte[] iv = new byte[16];
            FileInputStream temp = new FileInputStream(in);
            temp.read(iv);
            parameterSpec = new IvParameterSpec(iv);
        } catch (IOException e) {
            e.printStackTrace();
            return;
        }
```

```
        try {
            cipher.init(Cipher.DECRYPT_MODE, key, parameterSpec);
        } catch (InvalidKeyException e) {
            e.printStackTrace();
            AlertDialog.showError("AES-128 file decryption error!", e.toString());
            return;
        } catch (InvalidAlgorithmParameterException e) {
            e.printStackTrace();
            return;
            //perform file decryption without initialization vector
        }
        byte[] buffer = new byte[2048];
        try {
            FileOutputStream fileOut = new FileOutputStream(out);
            FileInputStream fileIn = new FileInputStream(in);
            CipherOutputStream cipherOut = new CipherOutputStream(fileOut, cipher);
            while (fileIn.read(buffer, 0, buffer.length)!=-1) {
                cipherOut.write(buffer, 0, buffer.length);
            }
            fileIn.close();
            fileOut.close();
            cipherOut.close();
            AlertDialog.showInfo("File was successfully decrypted!");
        } catch (IOException e) {
            e.printStackTrace();
            AlertDialog.showError("AES-128 file decryption error!", e.toString());
        }
    }

    void generateKey() {
        try {
            KeyGenerator generator = KeyGenerator.getInstance("AES");
            generator.init(128);
            key = generator.generateKey();
            System.out.println("AES-128 key generated");
        } catch (NoSuchAlgorithmException e) {
            e.printStackTrace();
        }
    }

    private void generateIV() {
        SecureRandom secureRandom = new SecureRandom();
        byte[] iv = new byte[16];
        secureRandom.nextBytes(iv);
        ivParameterSpec = new IvParameterSpec(iv);
```

```
    void setKey(String stringKey) {
        key = new SecretKeySpec(stringKey.getBytes(), "AES");
        System.out.println("AES-128 key set");
    }

    void setKeyBase64(String keyBase64) {
        key = new SecretKeySpec(Base64.getDecoder().decode(keyBase64), "AES");
        System.out.println("AES-128 key set");
    }

    String getKey() {
        return Base64.getEncoder().encodeToString(key.getEncoded());
    }

    private boolean checkKey(String inputKey) {
        return true;
    }

}
```

Fig 10: AES 256 code

SHA256.Java:

```
1  package Hash;
2
3  import GUI.Main.AlertDialog;
11
12 class SHA256 {
13
14     private MessageDigest messageDigest;
15
16     SHA256() {
17         try {
18             messageDigest = MessageDigest.getInstance("SHA-256");
19         } catch (NoSuchAlgorithmException e) {
20             e.printStackTrace();
21         }
22     }
23
24     String getHashFromText(String text) {
25         messageDigest.update(text.getBytes());
26         byte[] digest = messageDigest.digest();
27
28         BigInteger bigInt = new BigInteger(1, digest);
29         StringBuilder sha256Hex = new StringBuilder(bigInt.toString(16));
30
31         while( sha256Hex.length() < 32 ){
32             sha256Hex.insert(0, "0");
33         }
34
35         return sha256Hex.toString();
36     }
37
38     String getHashFromFile(File file) {
39         try {
40             FileInputStream fis = new FileInputStream(file);
41             byte[] buffer = new byte[1024];
42             int read;
43             while ((read = fis.read(buffer)) != -1) {
44                 messageDigest.update(buffer, 0, read);
45             }
46             fis.close();
47             byte[] digest = messageDigest.digest();
48
49             StringBuilder sb = new StringBuilder();
50             for (byte b : digest) {
51                 sb.append(Integer.toString((b & 0xff) + 0x100, 16).substring(1));
52             }
53             return sb.toString();
54         } catch (IOException e) {
55             e.printStackTrace();
56             AlertDialog.showError("Error getting SHA256 hash!", e.getMessage());
57         }
58         return null;
59     }
60
61 }
62
```

Fig 11: SHA256 code for Hashing

Output:

FrontPage:



Fig 12: Front Page

SideMenu:

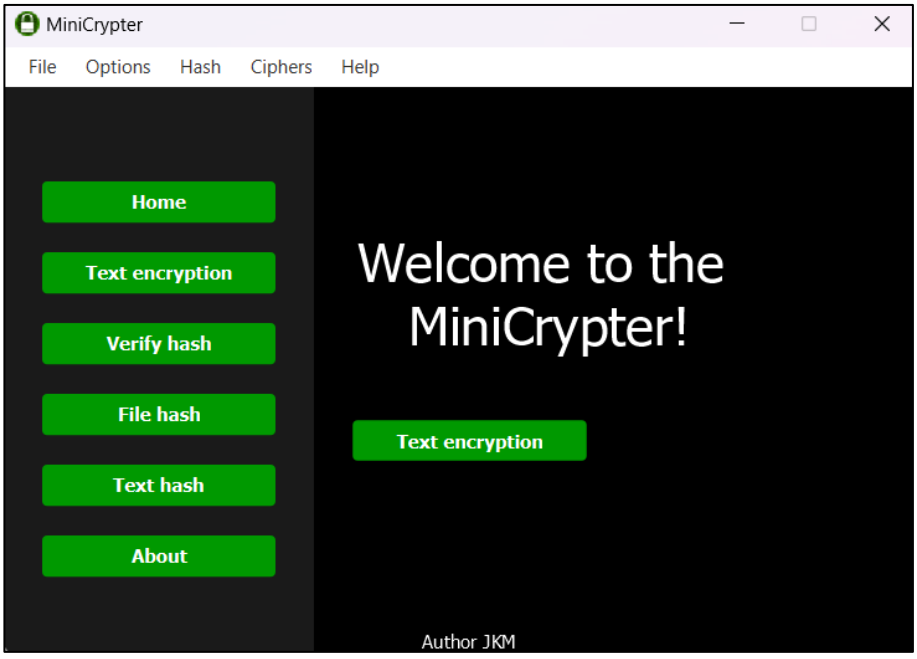


Fig 13: Side Menu

A view of available Hash Algorithms:

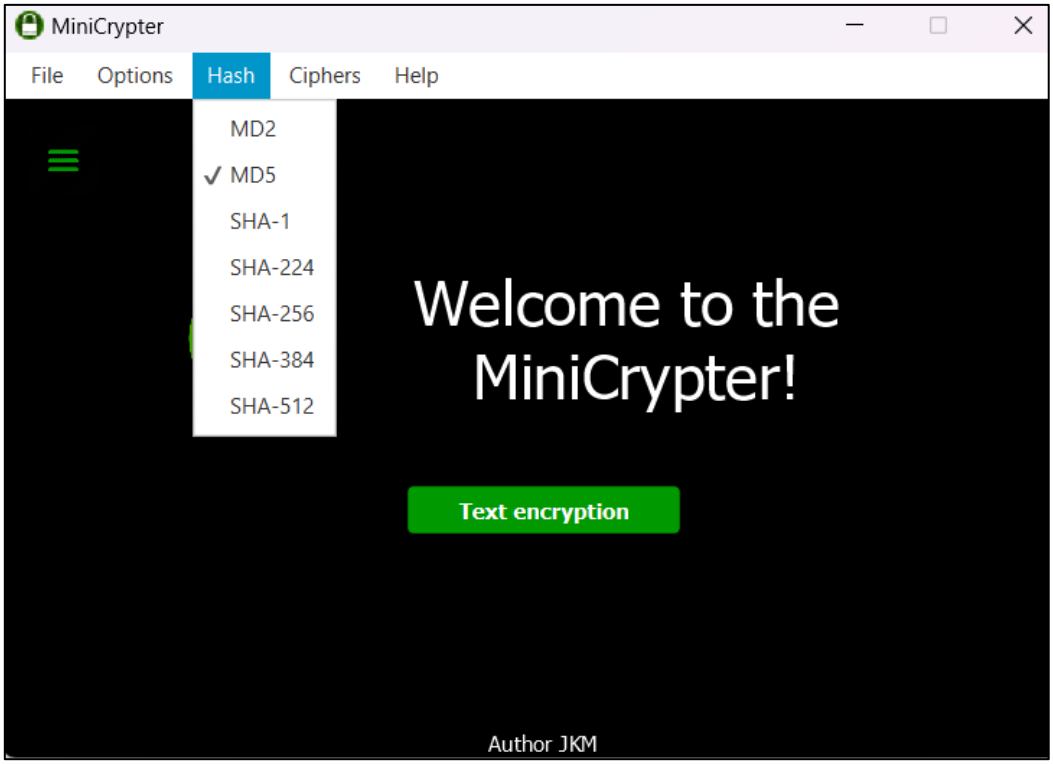


Fig 14: Available Hash Algorithms

A view of available Encryption/Decryption Algorithms:

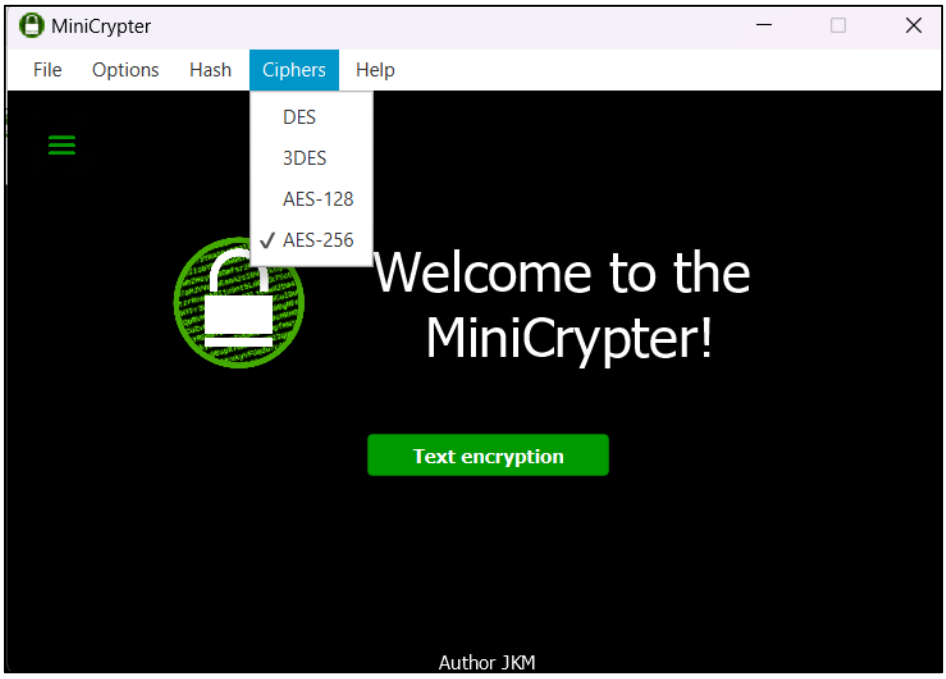


Fig 15: Available Encryption/Decryption Algorithms

Key Options:

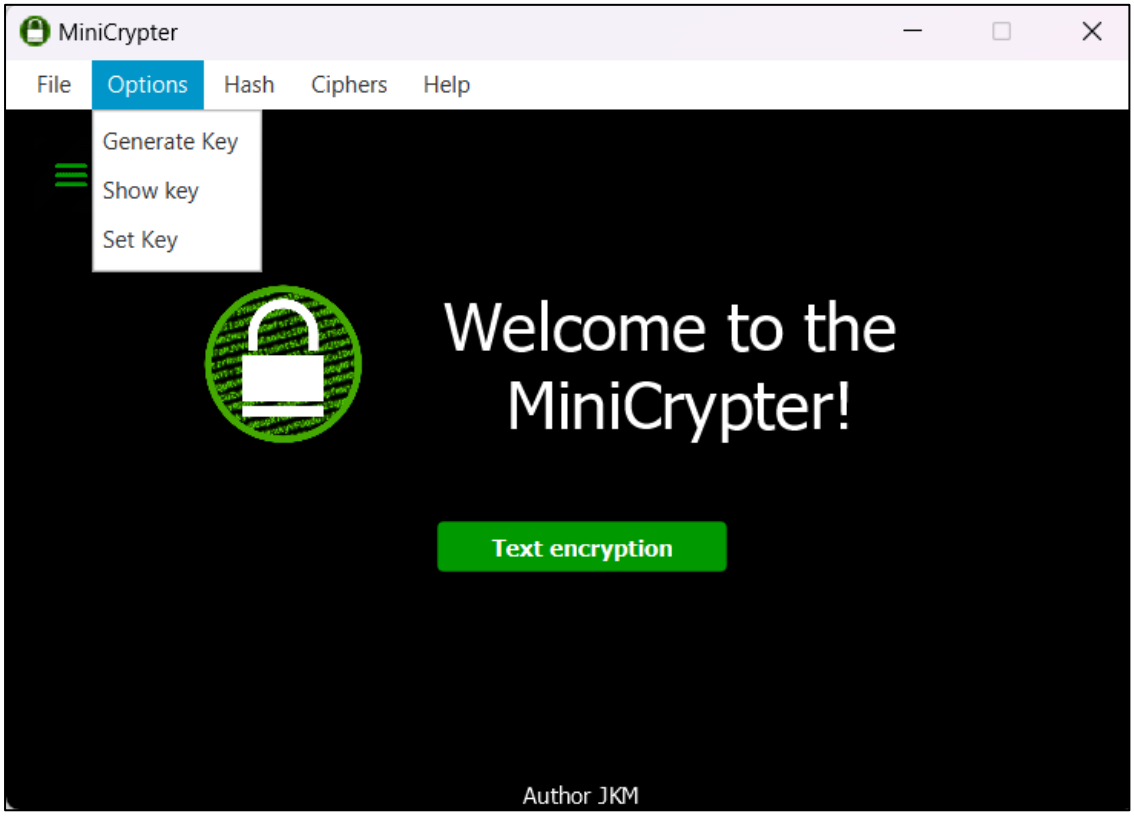


Fig 16: Key Options

Viewing the Generated Key:

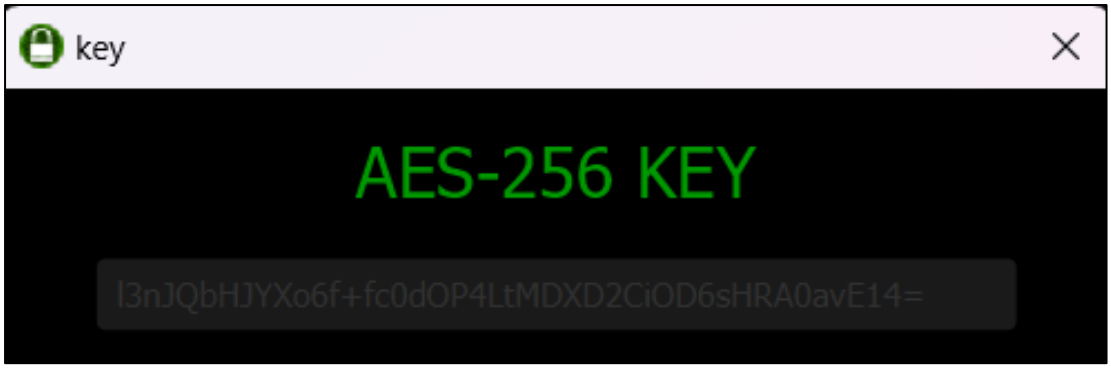


Fig 17: Viewing the Generated Key

Encryption decryption of a Text using AES256 algorithm:

On entering the text ‘Varsha’ and pressing the encrypt button, we get the decrypted string.

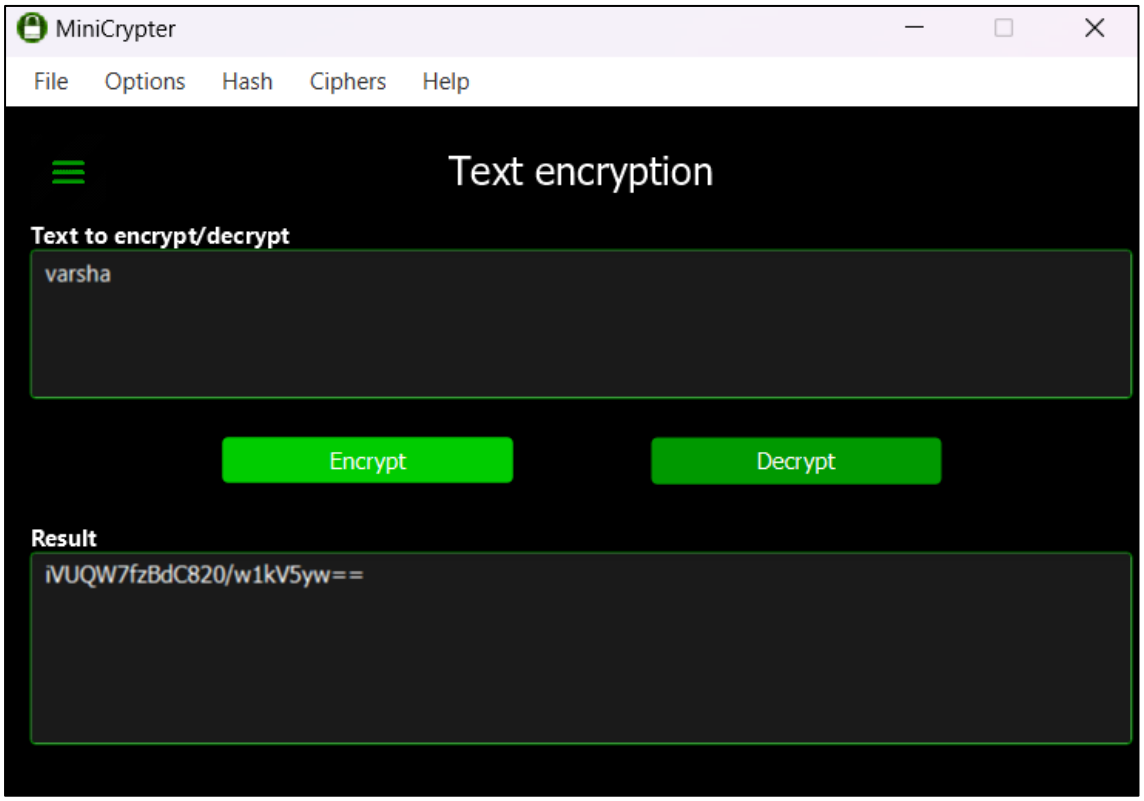


Fig 17: Text Encryption

By Entering the decrypted value in the input and pressing Decrypt button, we get the same encrypted value.

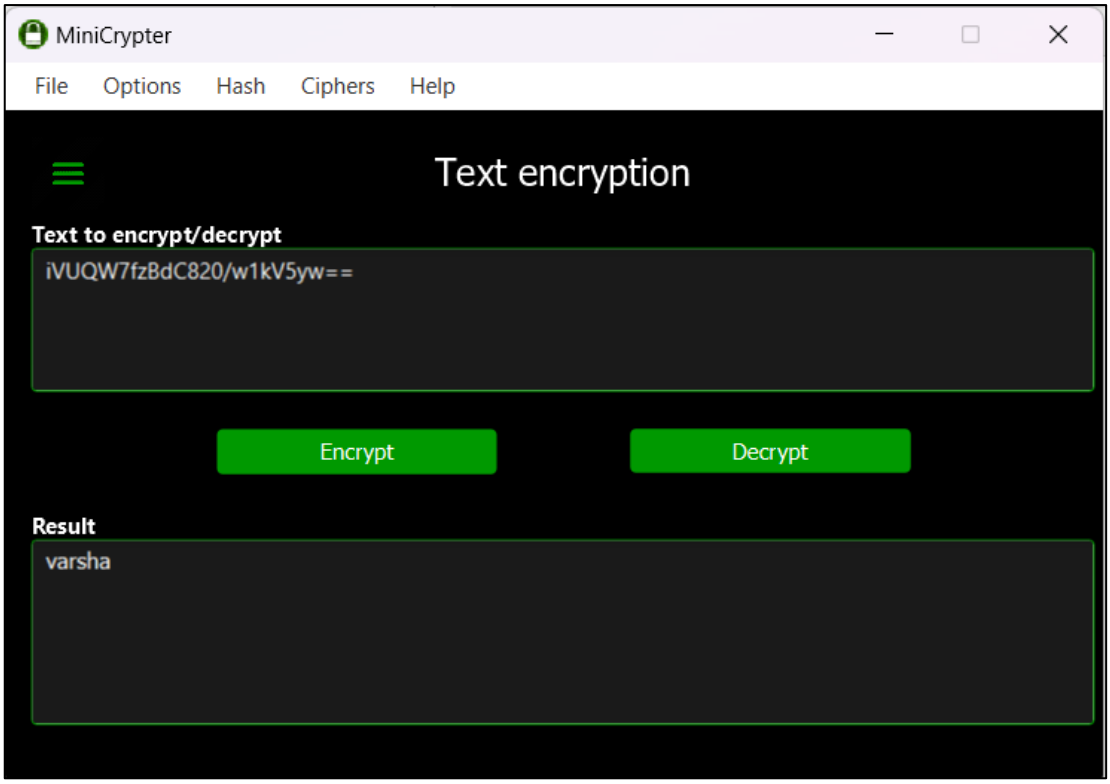


Fig 18: Text Decryption

When we give the wrong decrypted text, we would get an error.

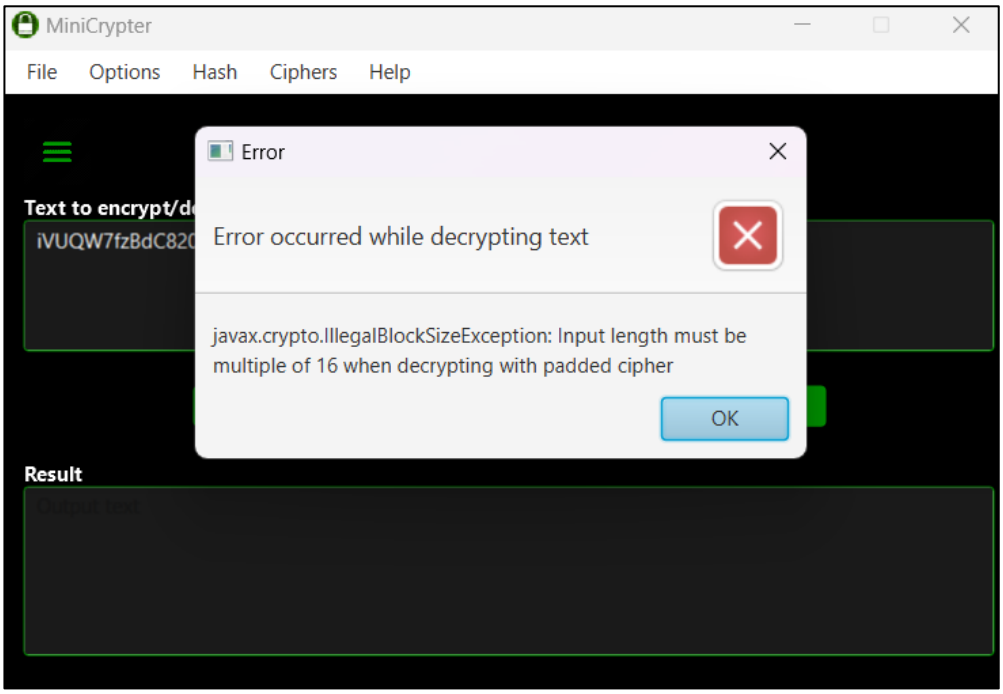


Fig 19: Text Decryption Error

Hashing a Text using SHA- 256

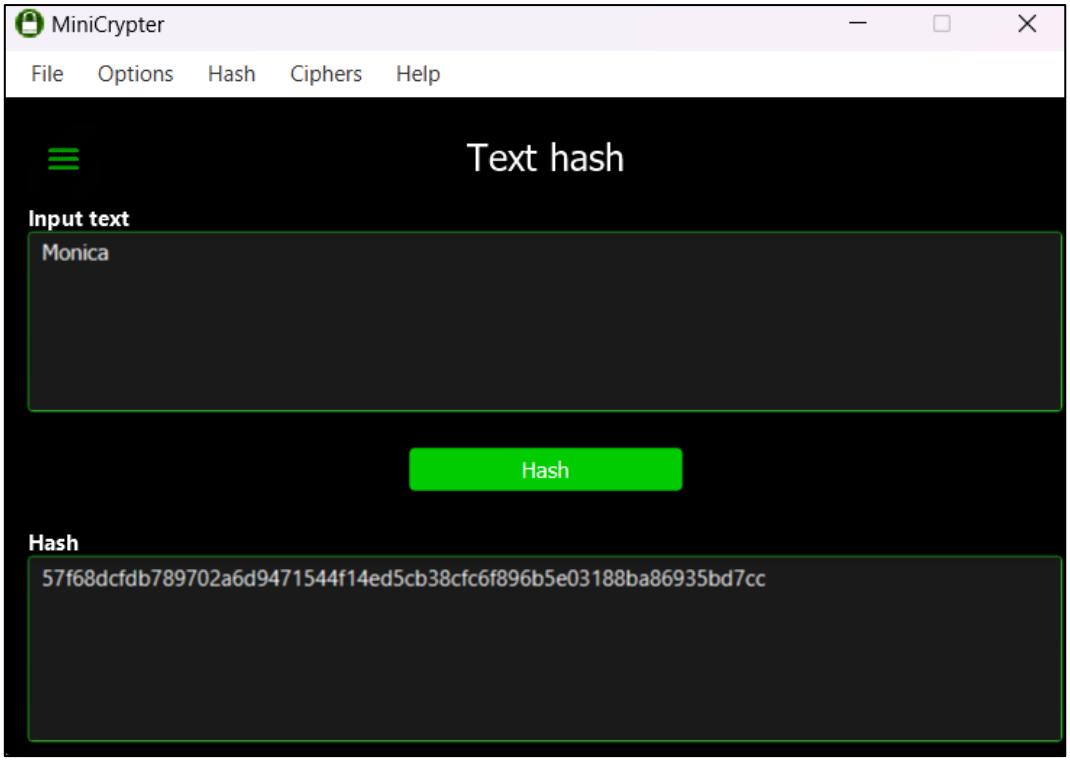


Fig 20: Text Hashing

Hashing a File using SHA256: The file contains the text ‘Monica’.

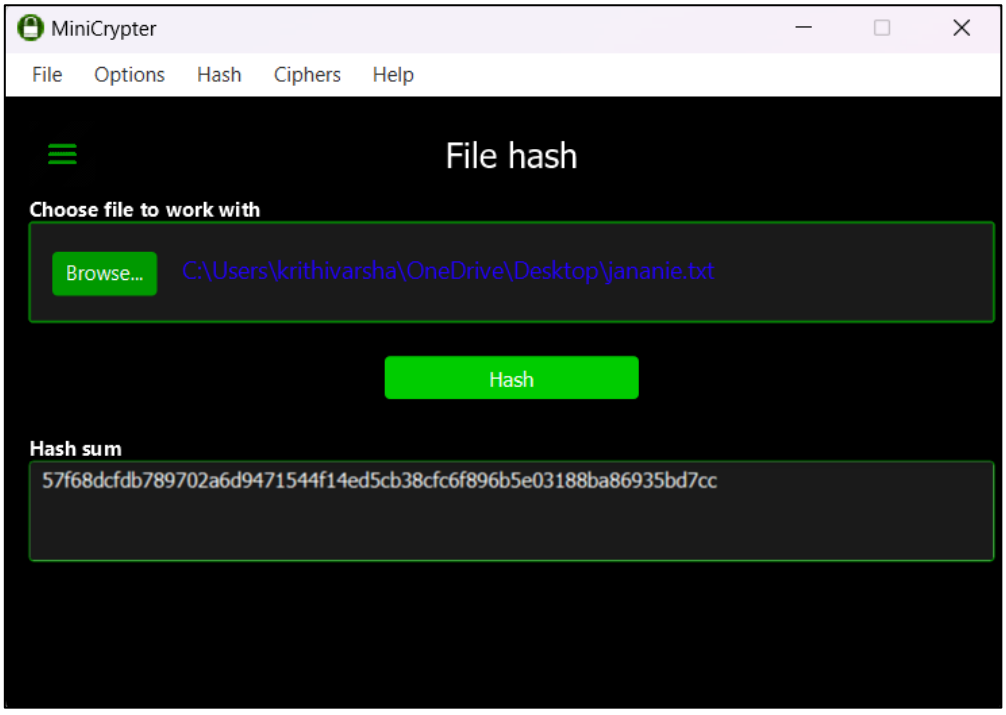


Fig 21: File Hashing

File Hash Verifier:

The Test-Hash file contains the text ‘Monica’. If we take the hash value of the Text ‘Monica’ that was generated in the Text Hash mode and paste in here, we get a correct mark symbolising that the content in the file and the text are the same from the same hash value produced.

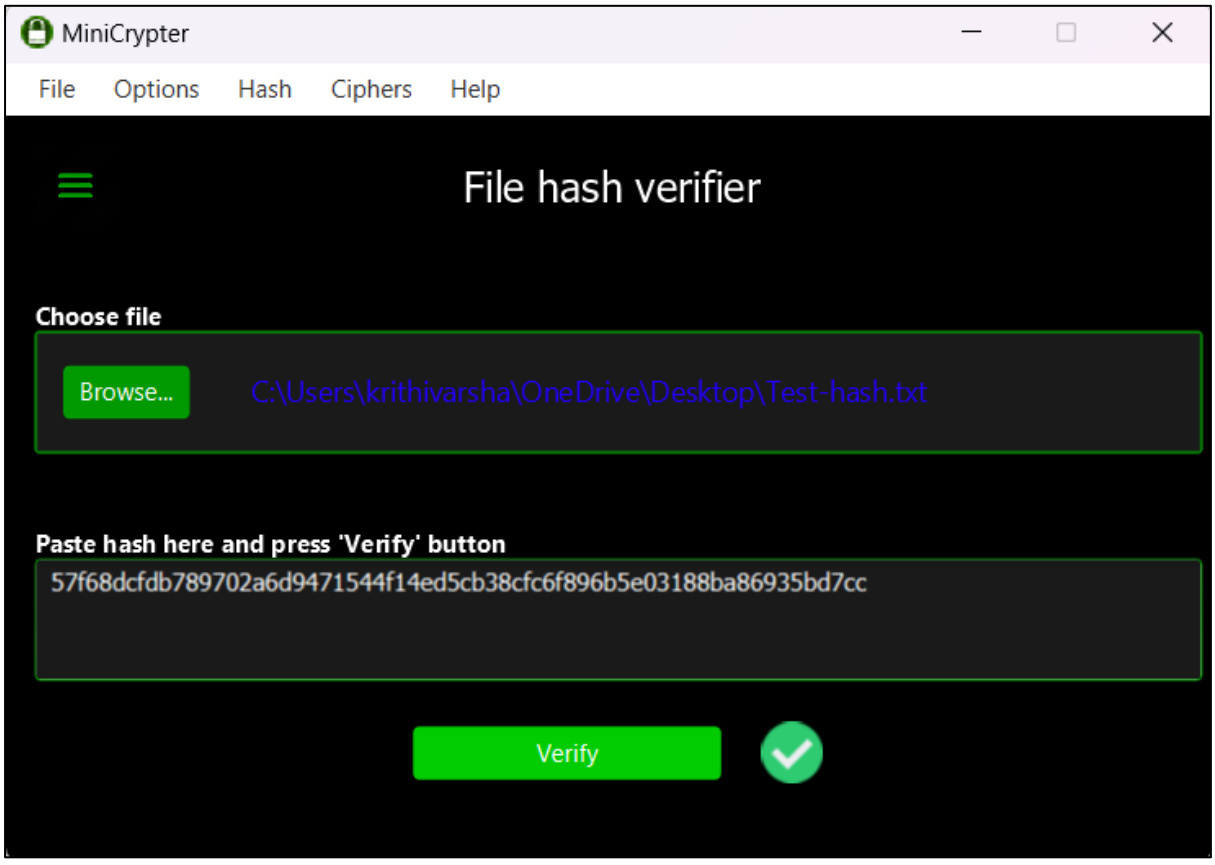


Fig 22: File hash Verifier

If we give a wrong hash value, then a cross mark would appear that symbolises that the contents of the file and the text do no match.

About Page:

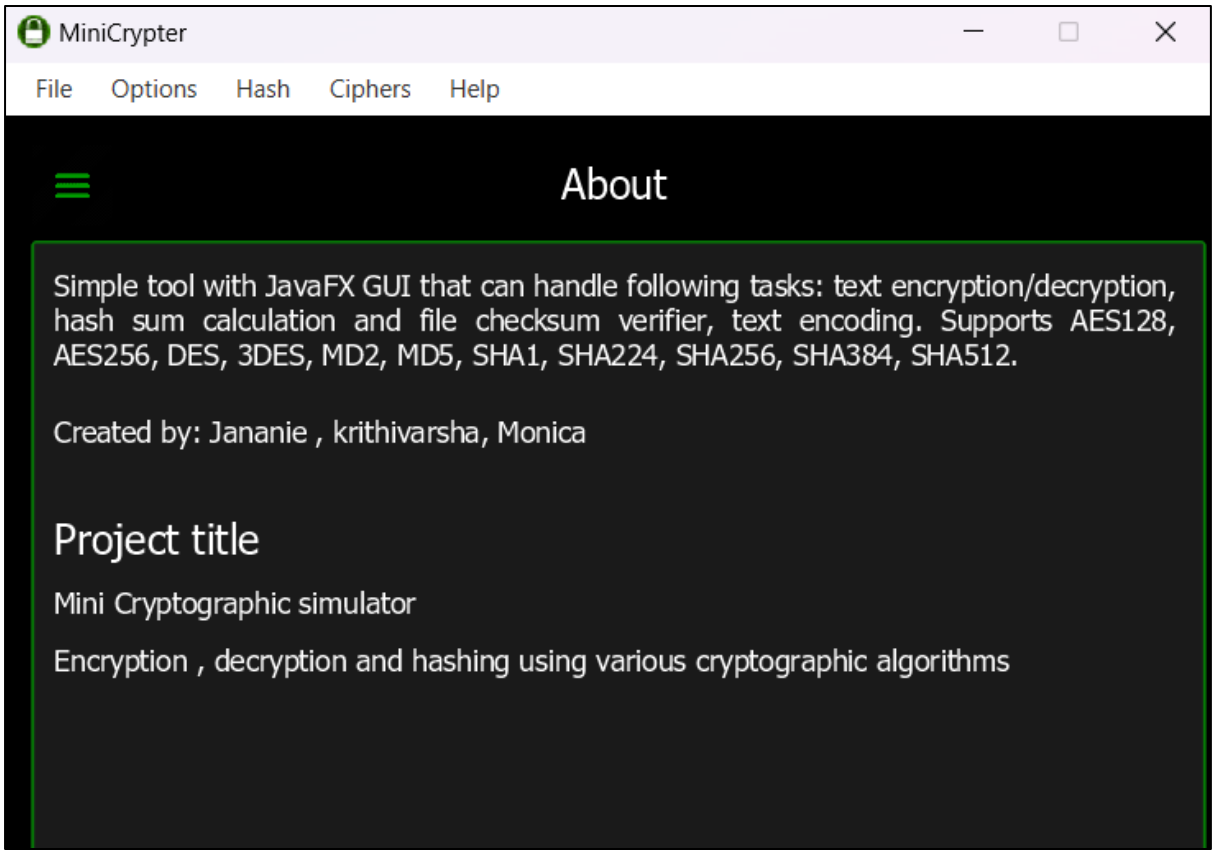


Fig 23: About Page